

Calcolatore di perimetri in Python

Calcolatore di perimetri in Python – Esercitazione W6D1

Corso: Epicode – Python Basics

Ambiente di sviluppo: Kali Linux, terminale + editor nano

File del progetto: perimetro.py

Obiettivo dell'esercizio

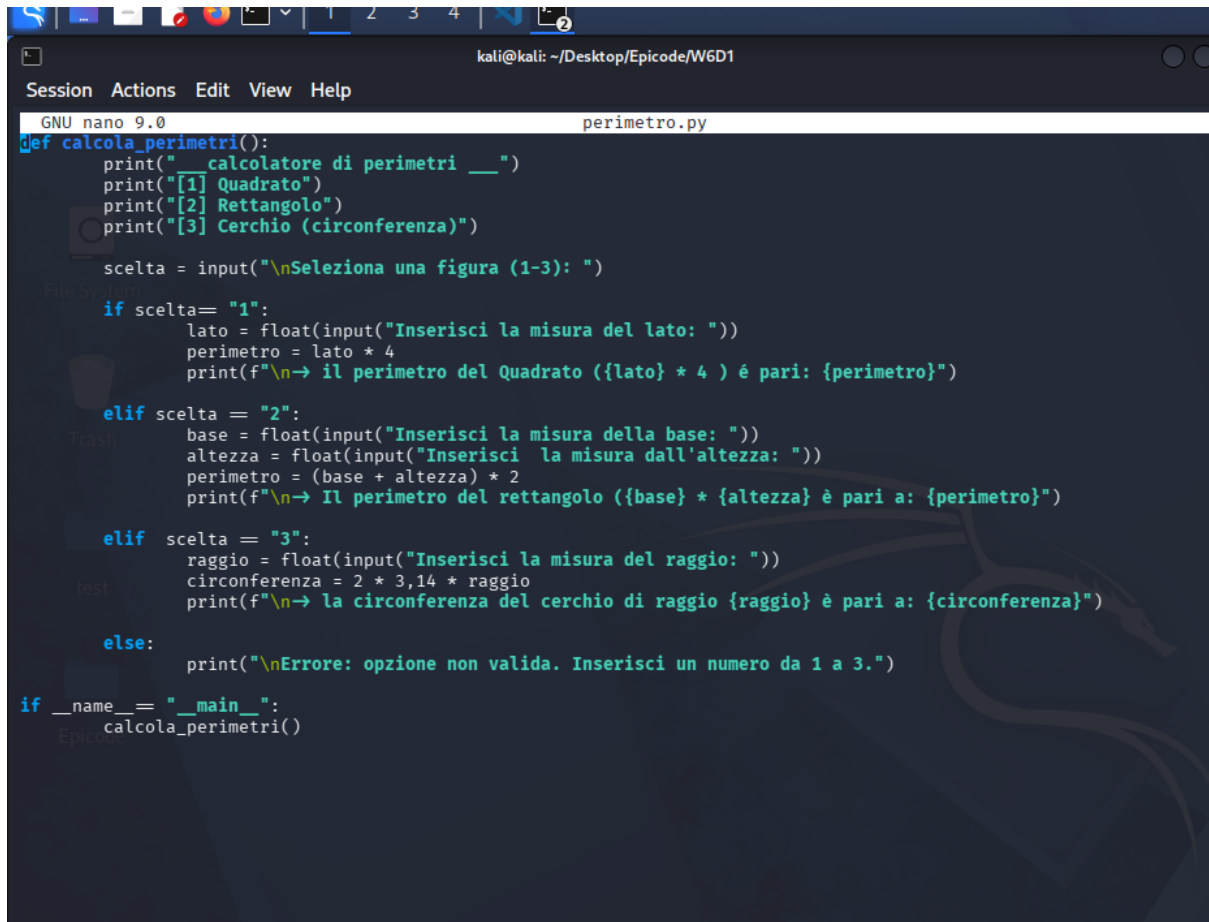
L'esercizio richiede la scrittura di un programma Python in grado di calcolare, in base alla scelta dell'utente, il perimetro (o la circonferenza) di una figura geometrica tra tre disponibili. Le figure scelte per lo svolgimento, insieme alle relative formule, sono:

- Quadrato $\rightarrow$ perimetro = lato $\times$ 4
- Rettangolo $\rightarrow$ perimetro = (base + altezza) $\times$ 2
- Cerchio $\rightarrow$ circonferenza = $2 \times \pi \times$ raggio

Il programma deve presentare un menu testuale, leggere da tastiera i valori necessari alla figura scelta e stampare a video il risultato del calcolo.

Struttura del programma

Il programma è organizzato in un'unica funzione, `calcola_perimetri()`, richiamata all'avvio dello script tramite il blocco standard `if __name__ == "__main__":`. La funzione stampa il menu delle tre opzioni, legge la scelta dell'utente con `input()`, e in base al valore selezionato (1, 2 o 3) richiede le misure necessarie ed esegue il calcolo corrispondente con un blocco `if / elif / else`.



```
GNU nano 9.0 perimetro.py
def calcola_perimetri():
    print("__calcolatore di perimetri __")
    print("[1] Quadrato")
    print("[2] Rettangolo")
    print("[3] Cerchio (circonferenza)")

    scelta = input("\nSeleziona una figura (1-3): ")

    if scelta == "1":
        lato = float(input("Inserisci la misura del lato: "))
        perimetro = lato * 4
        print(f"\n→ il perimetro del Quadrato ({lato} * 4 ) è pari: {perimetro}")

    elif scelta == "2":
        base = float(input("Inserisci la misura della base: "))
        altezza = float(input("Inserisci la misura dall'altezza: "))
        perimetro = (base + altezza) * 2
        print(f"\n→ Il perimetro del rettangolo ({base} * {altezza} è pari a: {perimetro}")

    elif scelta == "3":
        raggio = float(input("Inserisci la misura del raggio: "))
        circonferenza = 2 * 3,14 * raggio
        print(f"\n→ la circonferenza del cerchio di raggio {raggio} è pari a: {circonferenza}")

    else:
        print("\nErrore: opzione non valida. Inserisci un numero da 1 a 3.")

if __name__ == "__main__":
    calcola_perimetri()
```

Codice sorgente di perimetro.py, editor nano su Kali Linux

Esecuzione e test del programma

Il programma è stato eseguito più volte da terminale con il comando `python perimetro.py`, testando tutte e tre le opzioni del menu con valori diversi, per verificarne il corretto funzionamento.

```
kali@kali: ~/Desktop/Epicode/W6D1
Session Actions Edit View Help

(kali@kali)-[~/Desktop/Epicode/W6D1]
$ python perimetro.py
__calcolatore di perimetri __
[1] Quadrato
[2] Rettangolo
[3] Cerchio (circonferenza)

Seleziona una figura (1-3): 1
Inserisci la misura del lato: 2

→ il perimetro del Quadrato (2.0 * 4 ) é pari: 8.0

(kali@kali)-[~/Desktop/Epicode/W6D1]
$ python perimetro.py
__calcolatore di perimetri __
[1] Quadrato
[2] Rettangolo
[3] Cerchio (circonferenza)

Seleziona una figura (1-3): 2
Inserisci la misura della base: 1
Inserisci la misura dall'altezza: 1

→ Il perimetro del rettangolo (1.0 * 1.0 è pari a: 4.0

(kali@kali)-[~/Desktop/Epicode/W6D1]
$ python perimetro.py
__calcolatore di perimetri __
[1] Quadrato
[2] Rettangolo
[3] Cerchio (circonferenza)

Seleziona una figura (1-3): 1
Inserisci la misura del lato: 3

→ il perimetro del Quadrato (3.0 * 4 ) é pari: 12.0

(kali@kali)-[~/Desktop/Epicode/W6D1]
$ python perimetro.py
__calcolatore di perimetri __
[1] Quadrato
[2] Rettangolo
[3] Cerchio (circonferenza)

Seleziona una figura (1-3): 2
Inserisci la misura della base: 3
Inserisci la misura dall'altezza: 3

→ Il perimetro del rettangolo (3.0 * 3.0 è pari a: 12.0
```

Esecuzione delle opzioni Quadrato e Rettangolo con diversi valori di input

```
(kali㉿kali)-[~/Desktop/Epicode/W6D1]
└─$ python perimetro.py
__calcolatore di perimetri __
[1] Quadrato
[2] Rettangolo
[3] Cerchio (circonferenza)

Seleziona una figura (1-3): 3
Inserisci la misura del raggio: 2

→ la circonferenza del cerchio di raggio 2.0 è pari a: (6, 28.0)

(kali㉿kali)-[~/Desktop/Epicode/W6D1]
└─$ python perimetro.py
__calcolatore di perimetri __
[1] Quadrato
[2] Rettangolo
[3] Cerchio (circonferenza)

Seleziona una figura (1-3): 3
Inserisci la misura del raggio: 1

→ la circonferenza del cerchio di raggio 1.0 è pari a: (6, 14.0)

(kali㉿kali)-[~/Desktop/Epicode/W6D1]
└─$
```

Esecuzione dell'opzione Cerchio con raggio 1 e raggio 2

Riepilogo dei risultati ottenuti

Figura	Input inserito	Risultato atteso	Risultato ottenuto	Esito
Quadrato	lato = 2	perimetro = 8.0	8.0	OK
Quadrato	lato = 3	perimetro = 12.0	12.0	OK
Rettangolo	base = 1, altezza = 1	perimetro = 4.0	4.0	OK
Rettangolo	base = 3, altezza = 3	perimetro = 12.0	12.0	OK
Cerchio	raggio = 2	circonferenza = 12.56	(6, 28.0)	ERRORE
Cerchio	raggio = 1	circonferenza = 6.28	(6, 14.0)	ERRORE

Problemi riscontrati

Durante lo sviluppo sono stati corretti diversi errori di sintassi e di battitura nelle prime versioni del codice (nomi di variabili e funzioni non coerenti tra definizione e utilizzo, virgola mancante in un metodo, nome errato dell'eccezione ValueError, indentazione non uniforme). Una volta corretti questi

errori, il programma è stato eseguito correttamente per le opzioni Quadrato e Rettangolo, con risultati coerenti con i valori attesi (vedi tabella al punto 3.1).

Testando invece l'opzione Cerchio è stata rilevata un'anomalia: il risultato stampato non è un numero ma una coppia di valori tra parentesi, ad esempio (6, 28.0) per un raggio di 2. La causa è la formula usata nel codice:

```
Circonferenza = 2 * 3,14 * raggio
```

Il valore di pi greco è stato scritto con la virgola (3,14) invece che con il punto (3.14), come richiede la sintassi Python per i numeri decimali. Python non interpreta "3,14" come un numero: la virgola viene letta come separatore, e l'espressione `2 * 3,14 * raggio` viene calcolata come una tupla di due elementi, `(2*3, 14*raggio)`, che è esattamente il risultato anomalo osservato a video. La correzione consiste nel sostituire la virgola con il punto:

```
circonferenza = 2 * 3.14 * raggio
```

oppure, in modo più preciso:

```
import math
```

```
circonferenza = 2 * math.pi * raggio
```

Conclusioni

L'esercizio ha permesso di consolidare l'uso di strutture di controllo (if / elif / else), input da tastiera, conversione di tipo con `float()`, e formattazione dell'output con le f-string. Le opzioni Quadrato e Rettangolo funzionano correttamente e restituiscono risultati coerenti con le formule richieste. È stato individuato un bug nell'opzione Cerchio dovuto all'uso della virgola al posto del punto decimale nel valore di pi greco; la correzione proposta al punto 4 risolve il problema. Si consiglia di applicare la correzione e ripetere il test dell'opzione Cerchio prima della consegna definitiva.